CSE 211 — Atif Sir

Term-Final question bank with solutions

53 questions from the 2017-18, 2019-20, Jan-2020, 2021-22, 2022-23 and 2023-24 papers, grouped by the practice-tracker tabs and lessons. Each entry has a header line from the tracker (number, exam year, question number, marks, priority, lesson), followed by the question and its worked solution. Where the same question was asked in several papers, the stacked cards share one solution and later entries point back to it.

Priority: High = 2019-20 onward; Medium = the question type repeats in another paper; Low = appears once.

Tab	Questions
1 Intro + CNF	7
2 PDA	10
4 TM Techniques	6
5 TM Variants	6
6 Undecidability	7
7 Complexity	17

Solutions are taken from the CSE 211 Theory of Computation Topic-wise Solved Questions set (credited in that book to Mahdi Hasan, written with Claude Opus 5.5). Check any answer against your lecture notes and the note PDFs in the notes folder before relying on it in the exam.

1 Intro + CNF — Lectures 1–2 — Introduction, CFG simplification, CNF, CYK

Notes: CSE211_Atif_Intro_CNF_Notes.pdf

1. 2021-22 · Q5 · 15 marks · ★★★ High · Trace · L3

2021–2022 • Q5 Source M47 — MARKS 15

Convert to CNF, showing conversion steps: $S \to A1A \mid B$; $A \to 0A \mid 1A \mid \varepsilon$; $B \to 1$.

SOLUTION

We follow the order of the slides: ε-productions, unit productions, useless symbols, then the CNF steps.

(i) Eliminate ε-productions. Nullable: A ($A \to \varepsilon$). S and B are not nullable (every body of S contains 1 or B, and $B \to 1$). For each body, add every version with occurrences of A deleted, then drop $A \to \varepsilon$:

$$S \to A1A \mid 1A \mid A1 \mid 1 \mid B, \qquad A \to 0A \mid 1A \mid 0 \mid 1, \qquad B \to 1.$$

S is not nullable, so $\varepsilon \notin L$ and no $S \to \varepsilon$ is needed.

(ii) Eliminate unit productions. The only unit production is $S \to B$. Unit pairs: $(S,S), (A,A), (B,B)$ and (S,B). For (S,B) add $S \to 1$ (the non-unit body of B; already present), then remove $S \to B$:

$$S \to A1A \mid 1A \mid A1 \mid 1, \qquad A \to 0A \mid 1A \mid 0 \mid 1, \qquad B \to 1.$$

(iii) Eliminate useless symbols. Generating: $0, 1, A, S, B$ (all). Reachable from S: $S, A, 0, 1$; B is no longer reachable, so $B \to 1$ is deleted.

(iv)(a) Bodies of length ≥ 2 only of variables. Introduce $Y \to 1$, $Z \to 0$:

$$S \to AYA \mid YA \mid AY \mid 1, \qquad A \to ZA \mid YA \mid 0 \mid 1.$$

(iv)(b) Cascade bodies of length ≥ 3. $S \to AYA$ becomes $S \to AC_1$, $C_1 \to YA$.

Answer. $S \to AC_1 \mid YA \mid AY \mid 1$; $C_1 \to YA$; $A \to ZA \mid YA \mid 0 \mid 1$; $Y \to 1$; $Z \to 0$.

(The language is "strings containing at least one 1". Renaming C_1 as B gives exactly the grammar of 2022–2023 Q7 above.)

2. 2017-18 · Q7(a) · 8 marks · ★★ Medium · Trace · L3

2017–2018 • Q7(a–b) Source M34 — MARKS (a) 8; (b) 7

(a) Convert $S \to A1A \mid B$; $A \to 0A \mid 1A \mid \varepsilon$; $D \to 00$ into CNF, showing all steps (0 and 1 are terminals). (b) Use CYK to determine whether 01101 belongs to the resulting grammar.

SOLUTION

NOTE B has no productions and D is never used. We follow the grammar exactly as printed: B is non-generating and D is unreachable, so both disappear. (If B was meant to be D, step (ii) adds $S \to 00$, i.e. $S \to ZZ$ at the end; this does not affect part (b), since $01101 \neq 00$.)

(a) Order of the slides:

(i) *ε-productions.* Nullable: A only. $S \to A1A \mid 1A \mid A1 \mid 1 \mid B$; $A \to 0A \mid 1A \mid 0 \mid 1$; $D \to 00$. (S is not nullable, so no $S \to \varepsilon$.)

(ii) *Unit productions.* $S \to B$ is a unit production; the unit pair (S, B) adds nothing because B has no bodies. Removing $S \to B$: $S \to A1A \mid 1A \mid A1 \mid 1$.

(iii) *Useless symbols.* B is non-generating (no productions) and no longer occurs; D is generating but not reachable from S, so $D \to 00$ is deleted. Left: $S \to A1A \mid 1A \mid A1 \mid 1$; $A \to 0A \mid 1A \mid 0 \mid 1$.

(iv) *CNF steps.* (a) $Y \to 1$, $Z \to 0$: $S \to AYA \mid YA \mid AY \mid 1$; $A \to ZA \mid YA \mid 0 \mid 1$. (b) $S \to AYA$ becomes $S \to AC_1$, $C_1 \to YA$.

Answer. $S \to AC_1 \mid YA \mid AY \mid 1;\ C_1 \to YA;\ A \to ZA \mid YA \mid 0 \mid 1;\ Y \to 1;\ Z \to 0.$

(b) CYK on 01101 ($a_1 \cdots a_5 = 01101$; 1 is produced by A, S, Y, 0 by A, Z):

$X_{i,i+4}$	$\{A, \mathbf{S}\}$				
$X_{i,i+3}$	$\{A, \mathbf{S}\}$	$\{A, C_1, \mathbf{S}\}$			
$X_{i,i+2}$	$\{A, \mathbf{S}\}$	$\{A, C_1, \mathbf{S}\}$	$\{A, C_1, \mathbf{S}\}$		
$X_{i,i+1}$	$\{A, \mathbf{S}\}$	$\{A, C_1, \mathbf{S}\}$	$\{A, C_1, \mathbf{S}\}$	$\{A, \mathbf{S}\}$	
X_{ii}	$\{A, Z\}$	$\{A, \mathbf{S}, Y\}$	$\{A, \mathbf{S}, Y\}$	$\{A, Z\}$	$\{A, \mathbf{S}, Y\}$
	0	1	1	0	1

$S \in X_{15}$ (e.g. split $k = 1$: $A \in X_{11}$, $C_1 \in X_{25}$, rule $S \to AC_1$), so $01101 \in L(G)$ (as expected: it contains a 1).

3. 2023-24 · Q6 · 15 marks · ★★★ High · Trace · L4

2023–2024 • Q6(a–b) Source N03 — **MARKS (a) 10; (b) 5**

For the CNF grammar below: (a) use CYK to test 0010, showing the table; (b) describe $L(G)$ in simple English. $S \to BZ \mid CY \mid ZZ \mid YY$; $B \to ZA$; $C \to YA$; $A \to ZA \mid YA \mid 0 \mid 1$; $Z \to 0$; $Y \to 1$.

SOLUTION

(a) CYK for 0010

Here $a_1a_2a_3a_4 = 0010$. Bottom row: 0 is produced by A, Z and 1 by A, Y.

$X_{i,i+3}$	$\{A, B, \mathbf{S}\}$			
$X_{i,i+2}$	$\{A, B\}$	$\{A, B, \mathbf{S}\}$		
$X_{i,i+1}$	$\{A, B, \mathbf{S}\}$	$\{A, B\}$	$\{A, C\}$	
X_{ii}	$\{A, Z\}$	$\{A, Z\}$	$\{A, Y\}$	$\{A, Z\}$
	0	0	1	0

Sample entries: X_{12} ("00"): $Z \in X_{11}$, $A \in X_{22}$ give $A \to ZA$ and $B \to ZA$; $Z \in X_{11}$, $Z \in X_{22}$ give $S \to ZZ$; so $X_{12} = \{A, B, S\}$. X_{14}: the split $k = 3$ ("001" | "0") has $B \in X_{13}$, $Z \in X_{44}$, so $S \to BZ$ puts S in X_{14}. Since $S \in X_{14}$: $0010 \in L(G)$ ✓ (it starts and ends with 0).

(b) The language

$A \to ZA \mid YA \mid 0 \mid 1$ makes A generate every non-empty binary string. Hence $B = ZA$ generates $0\{0,1\}^+$ and $C = YA$ generates $1\{0,1\}^+$. So

$$S \to BZ:\ 0\,x\,0\ (|x| \geq 1), \quad S \to ZZ:\ 00, \quad S \to CY:\ 1\,x\,1, \quad S \to YY:\ 11.$$

Answer. $L(G)$ = binary strings of length at least 2 whose first and last symbols are the same.

4. 2022-23 · Q7 · 15 marks · ★★★ High · Trace · L4

2022–2023 • Q7 Source M52 **MARKS 15**

For this CNF grammar, use CYK to determine whether $0100 \in L(G)$, showing the table: $S \to AB \mid AY \mid YA \mid 1$; $A \to ZA \mid YA \mid 0 \mid 1$; $B \to YA$; $Z \to 0$; $Y \to 1$.

SOLUTION

(A = all non-empty strings, $B = 1\{0,1\}^+$, so $L(G)$ = all strings containing at least one 1.) Here $a_1 \cdots a_4 = 0100$.

$X_{i,i+3}$	$\{A, \mathbf{S}\}$			
$X_{i,i+2}$	$\{A, \mathbf{S}\}$	$\{A, B, \mathbf{S}\}$		
$X_{i,i+1}$	$\{A, \mathbf{S}\}$	$\{A, B, \mathbf{S}\}$	$\{A\}$	
X_{ii}	$\{A, Z\}$	$\{A, \mathbf{S}, Y\}$	$\{A, Z\}$	$\{A, Z\}$
	0	1	0	0

Top cell $X_{14} = \{A, S\}$ contains S (e.g. the split $k = 1$, "0" | "100": $A \in X_{11}$, $B \in X_{24}$, rule $S \to AB$). $0100 \in L(G)$.

5. 2020 (Jan term) · Q7 · 15 marks · ★★★ High · Trace · L4

January 2020 Term • Q7 Source M38 **MARKS 15**

For the following CNF grammar with start S, determine whether $10011 \in L(G)$ by CYK, showing the table. $S \to BZ \mid CY \mid ZZ \mid YY$; $B \to ZA$; $C \to YA$; $A \to ZA \mid YA \mid 0 \mid 1$; $Z \to 0$; $Y \to 1$.

SOLUTION

Same grammar as 2023–2024 Q6 above; $a_1 \cdots a_5 = 10011$.

$X_{i,i+4}$	$\{A, C, \mathbf{S}\}$				
$X_{i,i+3}$	$\{A, C, \mathbf{S}\}$	$\{A, B\}$			
$X_{i,i+2}$	$\{A, C\}$	$\{A, B\}$	$\{A, B\}$		
$X_{i,i+1}$	$\{A, C\}$	$\{A, B, \mathbf{S}\}$	$\{A, B\}$	$\{A, C, \mathbf{S}\}$	
X_{ii}	$\{A, Y\}$	$\{A, Z\}$	$\{A, Z\}$	$\{A, Y\}$	$\{A, Y\}$
	1	0	0	1	1

Sample entries: X_{12} ("10"): $Y \in X_{11}$, $A \in X_{22}$ give $A \to YA$, $C \to YA$; no body YZ exists, so $S \notin X_{12}$. X_{45} ("11"): $S \to YY$ applies. X_{15}: the split $k = 4$ ("1001" | "1") has $C \in X_{14}$, $Y \in X_{55}$, so $S \to CY$ gives $S \in X_{15}$. $10011 \in L(G)$ ✓ (it begins and ends with 1).

6. 2019-20 · Q7 · 15 marks · ★★★ High · Trace · L4

2019–2020 • Q7 Source M42 **MARKS 10 + 5 = 15**

For this CNF grammar with start S, use CYK to test 01100, showing the table. Describe $L(G)$ informally. $S \to BZ \mid CY \mid ZZ \mid YY$; $B \to ZA$; $C \to YA$; $A \to ZA \mid YA \mid 0 \mid 1$; $Z \to 0$; $Y \to 1$.

SOLUTION

Same grammar as 2023–2024 Q6 above.
CYK for 01100 ($a_1 \cdots a_5 = 01100$):

$X_{i,i+4}$	$\{A, B, \mathbf{S}\}$				
$X_{i,i+3}$	$\{A, B, \mathbf{S}\}$	$\{A, C\}$			
$X_{i,i+2}$	$\{A, B\}$	$\{A, C\}$	$\{A, C\}$		
$X_{i,i+1}$	$\{A, B\}$	$\{A, C, \mathbf{S}\}$	$\{A, C\}$	$\{A, B, \mathbf{S}\}$	
X_{ii}	$\{A, Z\}$	$\{A, Y\}$	$\{A, Y\}$	$\{A, Z\}$	$\{A, Z\}$
	0	1	1	0	0

Sample entries: X_{23} ("11"): $Y \in X_{22}$, $Y \in X_{33}$ give $S \to YY$; $Y \in X_{22}$, $A \in X_{33}$ give $A \to YA$, $C \to YA$; so $X_{23} = \{A, C, S\}$. X_{15}: the split $k = 4$ ("0110" | "0") has $B \in X_{14}$, $Z \in X_{55}$, so $S \to BZ$ gives $S \in X_{15}$. $01100 \in L(G)$ ✓.
Language (derived in 2023–2024 Q6(b) above):

Answer. $L(G)$ = binary strings of length at least 2 that begin and end with the same symbol.

7. 2017-18 · Q7(b) · 7 marks · ★★ Medium · Trace · L4

Same question — solution already given under #2.

2 PDA — Lecture 3 — Pushdown Automata, CFG–PDA equivalence, CFL pumping lemma

Notes: CSE211_Atif_PDA_Notes.pdf

8. 2023-24 · Q8 · 15 marks · ★★★ High · Design · L7

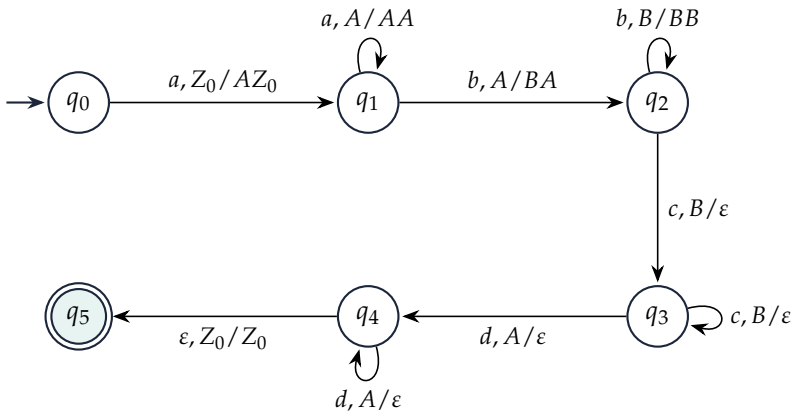
2023–2024 • Q8 Source N03 — MARKS 15

Design a PDA for $\{a^n b^m c^m d^n \mid n, m \geq 1\}$. Draw its transition diagram.

SOLUTION

Push an A per a and a B per b; pop the B's with c's, then the A's with d's. Because the B's sit above the A's, reading c with A on top means too many c's, and reading d with B on top means too few c's; neither situation has a move, so that branch dies. The first a $(a, Z_0/AZ_0)$ and the first b $(b, A/BA)$ have their own arcs, which enforces $n, m \geq 1$; the last arc needs Z_0 on top, i.e. every A has been matched by a d.

$$P = (\{q_0, \ldots, q_5\}, \{a, b, c, d\}, \{A, B, Z_0\}, \delta, q_0, Z_0, \{q_5\})$$



E.g. $aabcdd$: $(q_0, aabcdd, Z_0) \vdash (q_1, abcdd, AZ_0) \vdash (q_1, bcdd, AAZ_0) \vdash (q_2, cdd, BAAZ_0) \vdash (q_3, dd, AAZ_0) \vdash (q_4, d, AZ_0) \vdash (q_4, \varepsilon, Z_0) \vdash (q_5, \varepsilon, Z_0)$ — accepted.

9. 2022-23 · Q8 · 15 marks · ★★★ High · Design · L7

REPEATED ×2 Same question in 2 papers — stacked below, solved once.

2022–2023 • Q8 Source M52 — MARKS 15

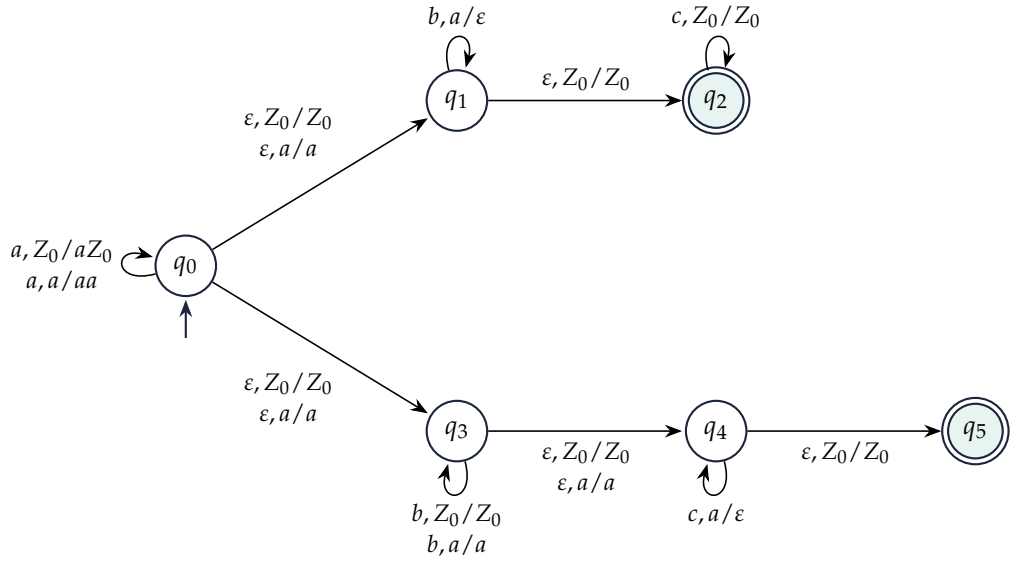
Design a PDA for $\{a^i b^j c^k \mid i, j, k \geq 0 \text{ and } (i = j \text{ or } i = k)\}$. Draw its transition diagram. Recall PDAs are nondeterministic.

January 2020 Term • Q8 Source M38 — MARKS 15

Design a PDA for $\{a^i b^j c^k \mid i, j, k \geq 0 \text{ and } (i = j \text{ or } i = k)\}$. Draw the transition diagram.

SOLUTION

Push the a's, then *guess* which count to compare them with. Upper branch: pop one a per b, check that Z_0 is on top again, then read the c's without touching the stack. Lower branch: read the b's without touching the stack, then pop one a per c and check Z_0.



$P = (\{q_0, \ldots, q_5\}, \{a, b, c\}, \{a, Z_0\}, \delta, q_0, Z_0, \{q_2, q_5\})$. If $i = j$ some computation takes the upper branch and accepts; if $i = k$ the lower branch accepts; otherwise every branch gets stuck or ends in a non-accepting state. Nondeterminism is essential here (the language is not deterministic context-free).

10. 2021-22 · Q7 · 15 marks · ★★★ High · Design · L7

REPEATED ×2 Same question in 2 papers — stacked below, solved once.

2021–2022 • Q7 Source M47 — MARKS 15

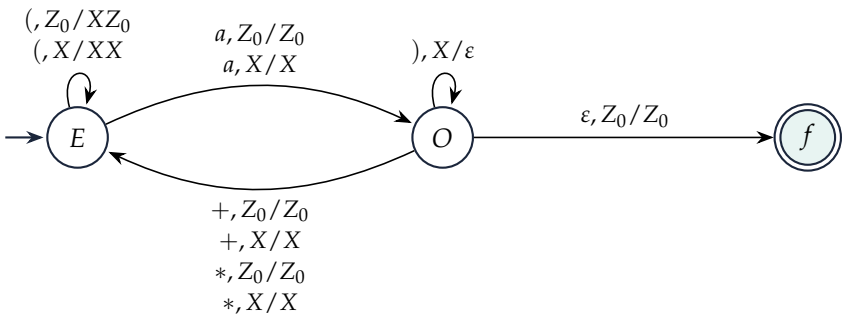
Construct a PDA for legal arithmetic expressions over $\{a, +, *, (,)\}$. Give its transition diagram. Valid: $a + a * a$, $((a + a) * (a + a)) + (a)$. Invalid: $aa * a$, $a{+}{+}a$, $+a$, $(a + a))$.

2019–2020 • Q8 Source M42 — MARKS 15

Design a PDA for legal arithmetic expressions over $\{a, +, *, (,)\}$. Give the transition diagram. (Same examples.)

SOLUTION

Idea. The finite control remembers whether we expect an *operand* (state E: start of input, after "(" or after an operator) or an *operator* (state O: after a or ")"). The stack counts open parentheses with a symbol X above Z_0.



$P = (\{E, O, f\}, \{a, +, *, (,)\}, \{X, Z_0\}, \delta, E, Z_0, \{f\})$.

- Adjacent variables ($aa*a$): in O there is no move on $a \Rightarrow$ reject.
- Adjacent operators ($a++a$): in E there is no move on $+$ or $*$.
- Initial operator ($+a$): the start state E has no move on $+$.
- Unbalanced ($(a+a))$): the second ")" finds Z_0 on top, and there is no move on ")" with Z_0 on top. A missing ")" leaves an X above Z_0, so the arc $O \to f$ (which needs Z_0 on top) is impossible.

Trace of $(a)+a$: $(E, (a)+a, Z_0) \vdash (E, a)+a, XZ_0) \vdash (O,)+a, XZ_0) \vdash (O, +a, Z_0) \vdash (E, a, Z_0) \vdash (O, \varepsilon, Z_0) \vdash (f, \varepsilon, Z_0)$ — accepted.

11. 2020 (Jan term) · Q8 · 15 marks · ★★★ High · Design · L7

Same question — solution already given under #9.

12. 2019-20 · Q8 · 15 marks · ★★★ High · Design · L7

Same question — solution already given under #10.

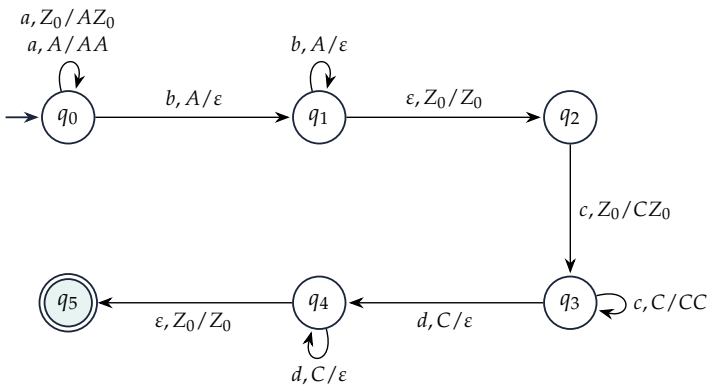
13. 2017-18 · Q8 · 15 marks · ★★ Medium · Design · L7

2017–2018 • Q8(a) Source M34 MARKS 15

Design a PDA recognizing $L = \{a^n b^n c^m d^m \mid n \geq 1,\ m \geq 1\}$. Draw its transition diagram.

SOLUTION

Match a's with b's, check that the stack is back at Z_0, then match c's with d's.



- $q_0 \to q_1$ needs an A on top, so at least one a was read ($n \geq 1$); in q_1 every b pops an A.
- $q_1 \xrightarrow{\varepsilon, Z_0/Z_0} q_2$ (stack unchanged) is possible only when Z_0 is on top, i.e. all A's are gone: $\#a = \#b$. A b read with Z_0 on top has no move (too many b's).

- $q_2 \to q_3$ forces at least one c; every c pushes a C, every d pops one; $q_4 \to q_5$ needs Z_0 on top, i.e. $\#c = \#d$.

14. 2019-20 · Q10(a) · 8 marks · ★★★ High · Proof · L8

REPEATED ×3 Same question in 3 papers — stacked below, solved once.

2019–2020 • Q10(a) Source M42 — **MARKS 8**

Show that languages accepted by final state by some PDA are exactly those accepted by empty stack by some PDA.

2015–2016 • Q6(b) Source M27 — **MARKS $10\frac{1}{3}$**

Given a PDA P_F accepting L by final state, can you always construct a PDA P_N accepting L by empty stack? If yes, give the construction and definitions of both P_F and P_N; otherwise justify.

2007–2008 • Q6 Source M05 — **MARKS 10**

Justify: a language has a PDA accepting by final state if and only if it has a PDA accepting by empty stack.

SOLUTION

SCOPE *2015–2016 Q6(b) asks only for the definitions and the direction $P_F \to P_N$ (the first two parts below); 2019–2020 Q10(a) and 2007–2008 Q6 ask for both directions.*

Definitions. A PDA is $P = (Q, \Sigma, \Gamma, \delta, q_0, Z_0, F)$. *Final-state* acceptance: $L(P) = \{w \mid (q_0, w, Z_0) \vdash^* (q, \varepsilon, \alpha),\ q \in F\}$. *Empty-stack* acceptance: $N(P) = \{w \mid (q_0, w, Z_0) \vdash^* (q, \varepsilon, \varepsilon)\}$ (any q).

Yes: from P_F (final state) to P_N (empty stack)

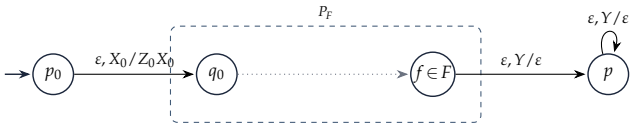
Given $P_F = (Q, \Sigma, \Gamma, \delta_F, q_0, Z_0, F)$ with $L = L(P_F)$, let $P_N = (Q \cup \{p_0, p\}, \Sigma, \Gamma \cup \{X_0\}, \delta_N, p_0, X_0)$:

(i) $\delta_N(p_0, \varepsilon, X_0) = \{(q_0, Z_0X_0)\}$ — put P_F's start symbol above a new bottom marker X_0. (X_0 prevents P_N from accidentally emptying its stack when P_F empties its own stack without accepting.)

(ii) All moves of P_F are copied: $\delta_N(q, a, Y) \supseteq \delta_F(q, a, Y)$.

(iii) For every $q \in F$ and $Y \in \Gamma \cup \{X_0\}$: $(p, \varepsilon) \in \delta_N(q, \varepsilon, Y)$ — when P_F is in an accepting state, P_N may start emptying.

(iv) For every $Y \in \Gamma \cup \{X_0\}$: $\delta_N(p, \varepsilon, Y) = \{(p, \varepsilon)\}$ — pop everything.



Correctness. If $w \in L(P_F)$: $(p_0, w, X_0) \vdash (q_0, w, Z_0X_0) \vdash^* (q, \varepsilon, \alpha X_0)$ with $q \in F$, then $\vdash (p, \varepsilon, \alpha X_0) \vdash^* (p, \varepsilon, \varepsilon)$. Conversely, P_N can empty its stack only by popping X_0, which happens only in p, entered only from an accepting state of P_F after simulating it on w; so $w \in L(P_F)$. Hence $N(P_N) = L$.

Conversely: from P_N (empty stack) to P_F (final state)

Given $P_N = (Q, \Sigma, \Gamma, \delta_N, q_0, Z_0)$, let $P_F = (Q \cup \{p_0, p_f\}, \Sigma, \Gamma \cup \{X_0\}, \delta_F, p_0, X_0, \{p_f\})$: $\delta_F(p_0, \varepsilon, X_0) = \{(q_0, Z_0X_0)\}$; copy all of δ_N; and for every $q \in Q$: $(p_f, \varepsilon) \in \delta_F(q, \varepsilon, X_0)$. Seeing X_0 on top means P_N has just emptied its own stack, so P_F moves to the accepting state p_f. *Correctness.* If $(q_0, w, Z_0) \vdash^*_{P_N} (q, \varepsilon, \varepsilon)$, then $(p_0, w, X_0) \vdash (q_0, w, Z_0X_0) \vdash^* (q, \varepsilon, X_0) \vdash (p_f, \varepsilon, \varepsilon)$; conversely p_f is entered only when X_0 is exposed, i.e. after P_N has emptied its stack. Thus $L(P_F) = N(P_N)$.

Answer. Both acceptance modes define exactly the same class of languages (the CFLs).

15. 2023-24 · Q7 · 15 marks · ★★★ High · Proof · L10

REPEATED ×4 Same question in 4 papers — stacked below, solved once.

2023–2024 • Q7 Source N03 — MARKS 5 + 10 = 15

Give a non-context-free language and prove it is not context-free using the CFL pumping lemma.

2021–2022 • Q8 Source M48 — MARKS 5 + 10 = 15

Give an example of a non-context-free language. Prove it is not context-free using the CFL pumping lemma.

2016–2017 • Q7(c) Source M31 — MARKS 10

Use the pumping lemma to show that $\{a^n b^n c^n \mid n \geq 0\}$ is not context-free.

2012–2013 • Q2(b) Source M17 — MARKS $13\frac{1}{3}$

Formally state the pumping lemma for CFLs. Use it to show that $\{a^n b^n c^n \mid n \geq 0\}$ is not context-free.

SOLUTION

SCOPE *Only 2012–2013 Q2(b) asks for the formal statement; the other papers need just the example and its proof.*

Statement (formal): For every CFL L there exists an integer $p \geq 1$ such that for all $w \in L$ with $|w| \geq p$ there exist u, v, x, y, z with $w = uvxyz$, $|vxy| \leq p$, $|vy| > 0$, and $\forall i \geq 0 :\ uv^i xy^i z \in L$.
Example: $L = \{a^n b^n c^n \mid n \geq 0\}$ is not context-free.
Proof. Assume, for contradiction, that L is context-free, and let p be its pumping length. Choose $w = a^p b^p c^p \in L$, $|w| = 3p \geq p$. Let $w = uvxyz$ be any split with $|vxy| \leq p$ and $|vy| > 0$. Pump with $i = 2$.

- *Case 1: v or y contains two different symbols.* Then uv^2xy^2z contains them out of order (e.g. $\ldots ab \ldots ab \ldots$), so it is not of the form $a^*b^*c^*$ and not in L.
- *Case 2: v and y each consist of one repeated symbol.* Since $|vxy| \leq p$ and each block has length p, vxy cannot touch both the a-block and the c-block. Hence v and y together contain at most two of the three letters, and at least one letter appears (as $|vy| > 0$). Pumping up, uv^2xy^2z has more of the (one or two) touched letters but still exactly p of the untouched letter, so the three counts are not equal: $uv^2xy^2z \notin L$.

Every split fails condition (3) for $i = 2$; this contradicts the pumping lemma. L is not context-free.

■

(Other standard examples: $\{ww \mid w \in \{0,1\}^*\}$, $\{a^{n^2}\}$, $\{a^i b^j c^k \mid i < j < k\}$.)

16. 2022-23 · Q9 · 15 marks · ★★★ High · Proof · L10

2022–2023 • Q9 Source M52 **MARKS 15**

State and prove the CFL pumping lemma. You may use the theorem: if a CNF parse tree yields terminal string w and its longest path has length n, then $|w| \leq 2^{n-1}$.

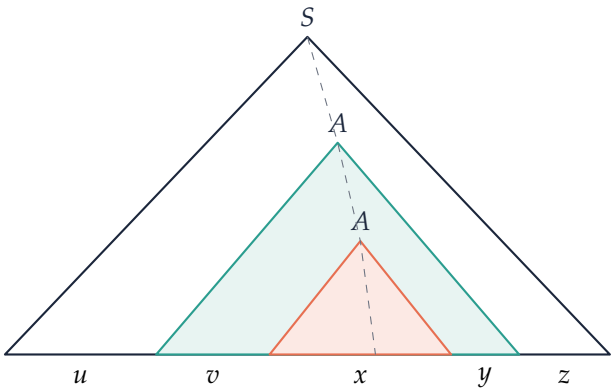
SOLUTION

Statement. If L is context-free, there is an integer $p \geq 1$ (the pumping length) such that every $w \in L$ with $|w| \geq p$ can be written $w = uvxyz$ with (1) $|vxy| \leq p$, (2) $|vy| > 0$, (3) $uv^i xy^i z \in L$ for every $i \geq 0$.

Proof. If $L = \emptyset$ or $L = \{\varepsilon\}$ any $p \geq 1$ works vacuously, so assume otherwise and let $G = (V, T, P, S)$ be a CNF grammar for $L - \{\varepsilon\}$ with $|V| = m$ variables. Let $p = 2^m$.

Take $w \in L$ with $|w| \geq p = 2^m$ (so $w \neq \varepsilon$ and $w \in L(G)$), and a parse tree for w. By the given theorem, if its longest path had length $\leq m$ then $|w| \leq 2^{m-1} < 2^m$. Hence the longest path has length $\geq m + 1$ (edges), i.e. at least $m + 2$ nodes; all but the last (a leaf) are variables, so the path contains at least $m + 1$ variable occurrences.

Look at the *last* (lowest) $m + 1$ variables on this longest path. By the pigeonhole principle two of them are equal: say $A_i = A_j = A$ with A_i above A_j. This gives the picture



where the subtree of the upper A yields vxy and the subtree of the lower A yields x. So $S \Rightarrow^* uAz$, $A \Rightarrow^* vAy$, $A \Rightarrow^* x$.

- **(3) Pumping.** Replacing the lower subtree by a copy of the upper one (repeatedly), or the upper subtree by the lower one, gives $S \Rightarrow^* uAz \Rightarrow^* uv^i Ay^i z \Rightarrow^* uv^i xy^i z$ for every $i \geq 0$; so $uv^i xy^i z \in L(G) \subseteq L$.
- **(2)** $|vy| > 0$**.** In CNF the upper A uses a rule $A \to BC$; one of B, C contains the lower A and the other yields a non-empty terminal string (a CNF grammar for $L - \{\varepsilon\}$ has no ε-rules), which is part of v or of y.
- **(1)** $|vxy| \leq p$**.** The path from the upper A down to the leaf, inside the longest path, has at most $m + 1$ edges (we chose A_i among the last $m + 1$ variables). It is a longest path of the upper A's subtree (a longer one there would give a longer path in the whole tree). By the theorem, the yield vxy of that subtree has length $\leq 2^{(m+1)-1} = 2^m = p$.

Hence $p = 2^m$ is a pumping length for L. ■

17. 2021-22 · Q8 · 15 marks · ★★★ High · Proof · L10

Same question — solution already given under #15.

4 TM Techniques — Lecture 5 — Programming techniques and bit-manipulation TMs

Notes: CSE211_Atif_TM_Techniques_Notes.pdf

18. 2023-24 · Q9 · 15 marks · ★★★ High · Design · L15

2023–2024 • Q9 Source N03 MARKS 15

Design a TM comparing binary w_1 and w_2. Initially the tape is $w_1 c w_2 c$. Write 0 after the second c if equal and 1 otherwise. You may assume equal lengths and use multiple tracks and state storage.

SOLUTION

Idea. Repeatedly take the leftmost unchecked bit b of w_1, store it in the state ($r_b = [r, b]$), check it off on the second track ($0 \mapsto x = [*, 0]$, $1 \mapsto y = [*, 1]$), walk to the leftmost unchecked bit of w_2 and compare. If they are equal, check that bit too and return; on a mismatch, walk to the cell after the second c and write 1. If every bit of w_1 is checked (the head reaches the first c), all pairs matched: write 0 after the second c.

$M = (\{q_0, r_0, r_1, s_0, s_1, k_1, k_2, e, n, z_0, z_1, h\}, \{0, 1, c\}, \{0, 1, c, x, y, B\}, \delta, q_0, B, \{h\})$ with δ:

State	0	1	x	y	c	B	meaning
$\to q_0$	(r_0, x, R)	(r_1, y, R)			(e, c, R)		next bit of w_1
r_0	$(r_0, 0, R)$	$(r_0, 1, R)$			(s_0, c, R)		carry 0 to w_2
r_1	$(r_1, 0, R)$	$(r_1, 1, R)$			(s_1, c, R)		carry 1 to w_2
s_0	(k_1, x, L)	$(n, 1, R)$	(s_0, x, R)	(s_0, y, R)			compare with 0
s_1	$(n, 0, R)$	(k_1, y, L)	(s_1, x, R)	(s_1, y, R)			compare with 1
k_1	$(k_1, 0, L)$	$(k_1, 1, L)$	(k_1, x, L)	(k_1, y, L)	(k_2, c, L)		back to first c
k_2	$(k_2, 0, L)$	$(k_2, 1, L)$	(q_0, x, R)	(q_0, y, R)			to last check in w_1
e			(e, x, R)	(e, y, R)	(z_0, c, R)		all equal
n	$(n, 0, R)$	$(n, 1, R)$	(n, x, R)	(n, y, R)	(z_1, c, R)		mismatch
z_0						$(h, 0, R)$	write 0
z_1						$(h, 1, R)$	write 1
$*h$							halt

h has no moves, so M halts there. In the slides' notation ($r_b = [r, b]$, $s_b = [s, b]$; an unchecked bit a is $[B, a]$, a checked one $[*, a]$, $c = [B, c]$, the blank $[B, B]$; $a, b \in \{0, 1\}$, $\bar{a} = 1 - a$):

Transition rules (numbered as in the slides):

1. $\delta(q_0, [B, a]) = ([r, a], [*, a], R)$ — check the next bit of w_1 and store it.
2. $\delta([r, a], [B, b]) = ([r, a], [B, b], R)$; $\delta([r, a], [B, c]) = ([s, a], [B, c], R)$ — go right past the first c.
3. $\delta([s, a], [*, b]) = ([s, a], [*, b], R)$ — skip the checked bits of w_2.
4. $\delta([s, a], [B, a]) = (k_1, [*, a], L)$ — equal bit: check it and turn back.
5. $\delta([s, a], [B, \bar{a}]) = (n, [B, \bar{a}], R)$ — different bit: mismatch.
6. $\delta(k_1, [t, b]) = (k_1, [t, b], L)$ for $t \in \{B, *\}$; $\delta(k_1, [B, c]) = (k_2, [B, c], L)$ — back over w_2 to the first c.
7. $\delta(k_2, [B, b]) = (k_2, [B, b], L)$; $\delta(k_2, [*, b]) = (q_0, [*, b], R)$ — back to the last checked bit of w_1; next round.
8. $\delta(q_0, [B, c]) = (e, [B, c], R)$; $\delta(e, [*, b]) = (e, [*, b], R)$; $\delta(e, [B, c]) = (z_0, [B, c], R)$ — w_1 used up, all bits equal: go past the second c.
9. $\delta(n, [t, b]) = (n, [t, b], R)$ for $t \in \{B, *\}$; $\delta(n, [B, c]) = (z_1, [B, c], R)$ — mismatch: go past the second c.
10. $\delta(z_0, [B, B]) = (h, [B, 0], R)$; $\delta(z_1, [B, B]) = (h, [B, 1], R)$ — write the answer and halt.

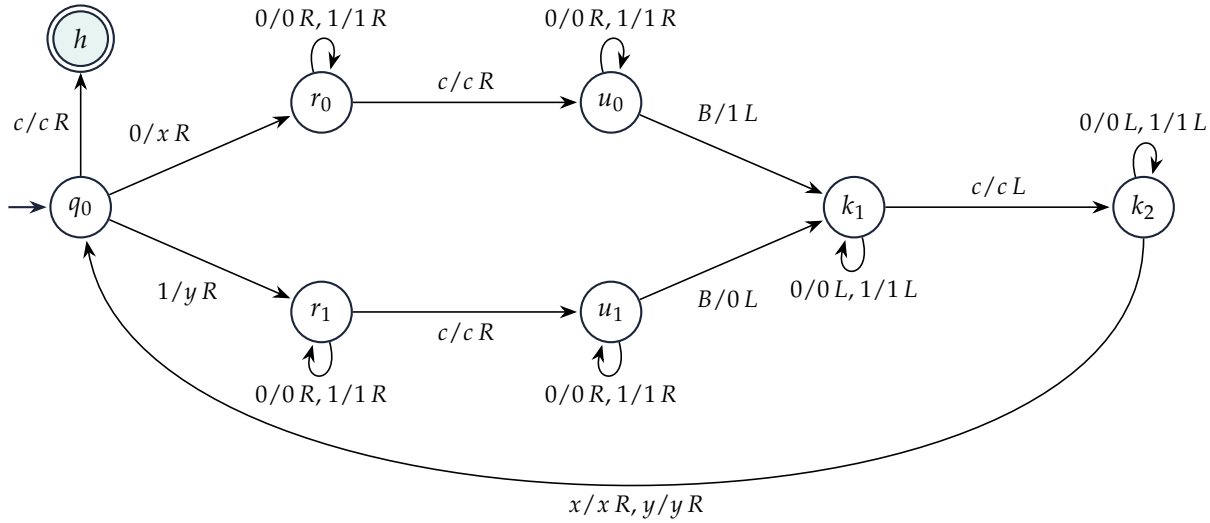
Example $w_1 = 10$, $w_2 = 11$: tape `10c11c`. Round 1: check $1 \mapsto y$ in w_1, carry 1 to w_2, its first unchecked bit is 1: equal, check it (y). Round 2: check $0 \mapsto x$, carry 0, the next unchecked bit of w_2 is 1 $\Rightarrow$ state n (that bit stays unchecked) $\Rightarrow$ z_1 writes 1. Final ID `yxcy1c1` $h\,B$, i.e. the tape is `yxcy1c1`. For `10c10c` the machine ends with `yxcyxc0`. (A last sweep can turn x, y back into $0, 1$ if required.)

2022–2023 • Q10 Source M52 **MARKS 15**

Design a TM computing bitwise NOT of binary w. Initially the tape is wc, with separator c. Insert the NOT of w in binary after c and terminate. Multiple tracks and state storage are allowed.

SOLUTION

For each bit of w from left to right: check it ($x = [*, 0]$, $y = [*, 1]$), carry it in the state to the first blank after c, write its complement there, come back.



Transition table ($r_b = [r, b]$, $u_b = [u, b]$; $x = [*, 0]$, $y = [*, 1]$):

State	0	1	c	x	y	B
$\to q_0$	(r_0, x, R)	(r_1, y, R)	(h, c, R)			
r_0	$(r_0, 0, R)$	$(r_0, 1, R)$	(u_0, c, R)			
r_1	$(r_1, 0, R)$	$(r_1, 1, R)$	(u_1, c, R)			
u_0	$(u_0, 0, R)$	$(u_0, 1, R)$				$(k_1, 1, L)$
u_1	$(u_1, 0, R)$	$(u_1, 1, R)$				$(k_1, 0, L)$
k_1	$(k_1, 0, L)$	$(k_1, 1, L)$	(k_2, c, L)			
k_2	$(k_2, 0, L)$	$(k_2, 1, L)$		(q_0, x, R)	(q_0, y, R)	
$*h$						

Transition rules ($a, b \in \{0, 1\}$, slides' notation):

1. $\delta(q_0, [B, a]) = ([r, a], [*, a], R)$ for $a \in \{0, 1\}$ — check the next bit and store it.
2. $\delta([r, a], [B, b]) = ([r, a], [B, b], R)$; $\delta([r, a], [B, c]) = ([u, a], [B, c], R)$ — go past c.
3. $\delta([u, a], [B, b]) = ([u, a], [B, b], R)$ — skip the bits already written.
4. $\delta([u, a], [B, B]) = (k_1, [B, \bar{a}], L)$ — write the complement $\bar{a} = 1 - a$.
5. $\delta(k_1, [B, b]) = (k_1, [B, b], L)$; $\delta(k_1, [B, c]) = (k_2, [B, c], L)$ — back to c.
6. $\delta(k_2, [B, b]) = (k_2, [B, b], L)$; $\delta(k_2, [*, b]) = (q_0, [*, b], R)$ — to the last checked bit; next round.
7. $\delta(q_0, [B, c]) = (h, [B, c], R)$ — every bit done: halt.

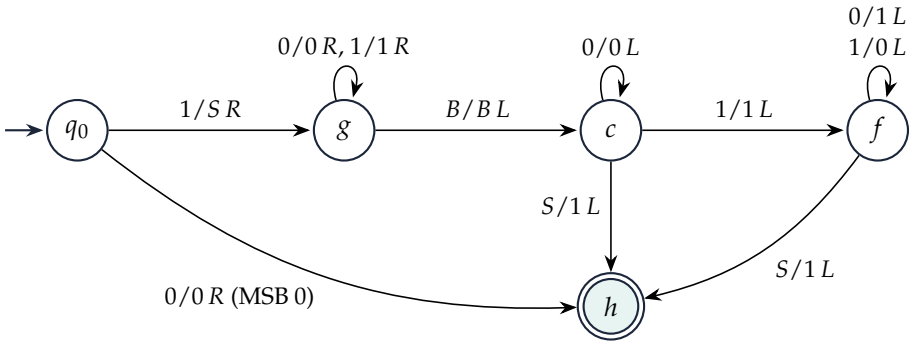
$r_b = [r, b]$ and $u_b = [u, b]$ remember that the bit read was b (so $\neg b$ is written). When q_0 meets c, all of w is processed: M enters h, which has no moves, and halts. Example $w = 10$: final tape `yxc01`.

2021–2022 • Q9 Source M48 MARKS 15

Construct a TM converting signed-magnitude binary w to two's complement. The leftmost bit is the MSB. If the MSB is 0, leave the number unchanged. If it is 1, scan from the LSB: copy zeros up to the first 1, copy that 1, and flip all remaining bits, leaving the MSB as 1. Finish with only the two's-complement number on the tape.

SOLUTION

Mark the sign bit with S (so the right-to-left scan knows where to stop), go to the right end, then implement "copy up to and including the first 1, then flip".



State	0	1	S	B
$\to q_0$	$(h,0,R)$	(g,S,R)		
g	$(g,0,R)$	$(g,1,R)$		(c,B,L)
c	$(c,0,L)$	$(f,1,L)$	$(h,1,L)$	
f	$(f,1,L)$	$(f,0,L)$	$(h,1,L)$	
$*h$				

Transition rules (numbered as in the slides):

1. $\delta(q_0,0)=(h,0,R)$ — MSB 0: the number is non-negative; halt.
2. $\delta(q_0,1)=(g,S,R)$ — MSB 1: mark it S.
3. $\delta(g,a)=(g,a,R)$ for $a\in\{0,1\}$; $\delta(g,B)=(c,B,L)$ — go to the LSB.
4. $\delta(c,0)=(c,0,L)$ — keep the trailing 0's.
5. $\delta(c,1)=(f,1,L)$ — keep the first 1, start flipping.
6. $\delta(f,0)=(f,1,L)$, $\delta(f,1)=(f,0,L)$ — flip the remaining bits.
7. $\delta(c,S)=(h,1,L)$, $\delta(f,S)=(h,1,L)$ — restore the sign bit 1 and halt.

- q_0: MSB 0 $\Rightarrow$ the number is non-negative: go to h (no moves), leaving the tape unchanged. MSB 1 $\Rightarrow$ replace it by S.
- g: run to the right end; c: leave the trailing 0's, leave the first 1 and switch to f; f: flip every bit until S; finally restore $S\to 1$ and halt in h.

Example 1101 (-5): $S101\to$ LSB 1 kept, then $0\to 1$, $1\to 0$: 1011, which is -5 in 4-bit two's complement ✓.

21. 2020 (Jan term) · Q9 · 15 marks · ★★★ High · Design · L15

January 2020 Term • Q9 Source M38 **MARKS 15**

Design a TM computing XOR of equal-length binary w_1 and w_2. Initially the tape contains w_1cw_2c. Finish with the XOR in binary after the second c.

SOLUTION

Same machine as 2019–2020 Q9 above, with $\odot$ = XOR. Only the two s_b rows change (XOR column of the operation table):

State	0	1	x	y
s_0	(t_0, x, R)	(t_1, y, R)	(s_0, x, R)	(s_0, y, R)
s_1	(t_1, x, R)	(t_0, y, R)	(s_1, x, R)	(s_1, y, R)

In the rule list of 2019–2020 Q9 only rule 4 changes: $\delta([s, a], [B, b]) = ([t, a \oplus b], [*, b], R)$.
Example: `10c11c` $\rightarrow$ round 1 writes $1 \oplus 1 = 0$, round 2 writes $0 \oplus 1 = 1$: final tape `yxcyyc01`, i.e. the XOR `01` after the second c. Time $O(n^2)$.

22. 2019-20 · Q9 · 15 marks · ★★★ High · Design · L15

2019–2020 • Q9 Source M42 **MARKS 15**

Design a TM computing AND of equal-length binary w_1 and w_2. Initially the tape is w_1cw_2c. Finish with their AND in binary after the second c. Multiple tracks and state storage are allowed.

SOLUTION

The machine below is written for a bitwise operation $\odot$; here $\odot$ = AND (the next two blocks reuse it with XOR and OR).
Algorithm. Repeat: (1) take the leftmost unchecked bit b of w_1, store it in the state ($r_b = [r, b]$) and check it ($x = [*, 0]$, $y = [*, 1]$); (2) go right past the first c to the leftmost unchecked bit b' of w_2, check it and store $o = b \odot b'$ in the state (t_o); (3) go right past the second c and past the result bits written so far to the first blank, write o; (4) return left to the last checked bit of w_1. When step (1) finds c instead of a bit, enter the halting state h.
$M = (Q, \{0, 1, c\}, \{0, 1, c, x, y, B\}, \delta, q_0, B, \{h\})$, $Q = \{q_0, r_0, r_1, s_0, s_1, t_0, t_1, u_0, u_1, k_1, k_2, k_3, h\}$:

State	0	1	x	y	c	B
$\rightarrow q_0$	(r_0, x, R)	(r_1, y, R)			(h, c, R)	
r_0	$(r_0, 0, R)$	$(r_0, 1, R)$			(s_0, c, R)	
r_1	$(r_1, 0, R)$	$(r_1, 1, R)$			(s_1, c, R)	
s_0	(t_0, x, R)	(t_0, y, R)	(s_0, x, R)	(s_0, y, R)		
s_1	(t_0, x, R)	(t_1, y, R)	(s_1, x, R)	(s_1, y, R)		
t_0	$(t_0, 0, R)$	$(t_0, 1, R)$			(u_0, c, R)	
t_1	$(t_1, 0, R)$	$(t_1, 1, R)$			(u_1, c, R)	
u_0	$(u_0, 0, R)$	$(u_0, 1, R)$				$(k_1, 0, L)$
u_1	$(u_1, 0, R)$	$(u_1, 1, R)$				$(k_1, 1, L)$
k_1	$(k_1, 0, L)$	$(k_1, 1, L)$			(k_2, c, L)	
k_2	$(k_2, 0, L)$	$(k_2, 1, L)$	(k_2, x, L)	(k_2, y, L)	(k_3, c, L)	
k_3	$(k_3, 0, L)$	$(k_3, 1, L)$	(q_0, x, R)	(q_0, y, R)		
$*h$						

h has no moves. The rows s_0, s_1 are those for $\odot$ = AND; in general $\delta(s_b, b') = (t_{b \odot b'}, \cdot, R)$, with $b \odot b'$ from the operation table:

b	b'	AND	XOR	OR
0	0	0	0	0
0	1	0	1	1
1	0	0	1	1
1	1	1	0	1

In the slides' notation ($r_b = [r,b]$, $s_b = [s,b]$, $t_o = [t,o]$, $u_o = [u,o]$; unchecked bit $[B,a]$, checked bit $[*,a]$, $c = [B,c]$, blank $[B,B]$; $a,b,o \in \{0,1\}$):

Transition rules (numbered as in the slides):

1. $\delta(q_0,[B,a]) = ([r,a],[*,a],R)$ — check the next bit a of w_1 and store it.
2. $\delta([r,a],[B,b]) = ([r,a],[B,b],R)$; $\delta([r,a],[B,c]) = ([s,a],[B,c],R)$ — go to w_2.
3. $\delta([s,a],[*,b]) = ([s,a],[*,b],R)$ — skip the checked bits of w_2.
4. $\delta([s,a],[B,b]) = ([t,a\odot b],[*,b],R)$ — check bit b of w_2 and store $o = a \odot b$ (here $a \wedge b$).
5. $\delta([t,o],[B,b]) = ([t,o],[B,b],R)$; $\delta([t,o],[B,c]) = ([u,o],[B,c],R)$ — go past the second c.
6. $\delta([u,o],[B,b]) = ([u,o],[B,b],R)$; $\delta([u,o],[B,B]) = (k_1,[B,o],L)$ — write o after the result bits so far.
7. $\delta(k_1,[B,b]) = (k_1,[B,b],L)$; $\delta(k_1,[B,c]) = (k_2,[B,c],L)$ — back to the second c.
8. $\delta(k_2,[t,b]) = (k_2,[t,b],L)$ for $t \in \{B,*\}$; $\delta(k_2,[B,c]) = (k_3,[B,c],L)$ — across w_2 to the first c.
9. $\delta(k_3,[B,b]) = (k_3,[B,b],L)$; $\delta(k_3,[*,b]) = (q_0,[*,b],R)$ — to the last checked bit of w_1; next round.
10. $\delta(q_0,[B,c]) = (h,[B,c],R)$ — w_1 used up: halt.

Example (AND): `10c11c` $\to$ round 1 writes $1 \wedge 1 = 1$, round 2 writes $0 \wedge 1 = 0$; then q_0 reads c and halts: final tape `yxcyyc10`, i.e. the AND `10` after the second c. (A final sweep can restore $x \to 0$, $y \to 1$.) The machine performs n rounds of $O(n)$ moves: $O(n^2)$ time.

23. 2017-18 · Q9 · 15 marks · ★★ Medium · Design · L15

2017–2018 • Q9 Source M34 — MARKS 15

Design a TM computing bitwise OR of equal-length binary M_1 and M_2. Initially the tape is $M_1\#M_2\#$. Finish with the binary OR after the second #.

SOLUTION

Same machine as 2019–2020 Q9 above, with $\odot$ = OR and the separator # in place of c (every c in the table becomes #). The two s_b rows (OR column of the operation table):

State	0	1	x	y
s_0	(t_0,x,R)	(t_1,y,R)	(s_0,x,R)	(s_0,y,R)
s_1	(t_1,x,R)	(t_1,y,R)	(s_1,x,R)	(s_1,y,R)

In the rule list of 2019–2020 Q9 rule 4 becomes $\delta([s,a],[B,b]) = ([t,a\vee b],[*,b],R)$, and $[B,\#]$ replaces $[B,c]$ everywhere.

Example: `10#01#` $\to$ round 1 writes $1 \vee 0 = 1$, round 2 writes $0 \vee 1 = 1$; q_0 then reads # and halts: final tape `yx#xy#11`, i.e. the OR `11` after the second #. Time $O(n^2)$.

5 TM Variants — Lecture 6 — Variants of Turing machines, TMs vs computers

Notes: CSE211_Atif_TM_Variants_Notes.pdf

24. 2023-24 · Q10 · 15 marks · ★★★ High · Theory · L16

REPEATED ×3 Same question in 3 papers — stacked below, solved once.

2023–2024 • Q10 Source N03 — MARKS 5 + 10 = 15

What is a multitape TM? Explain how a one-tape TM with multiple tracks and state storage simulates it.

2019–2020 • Q10(b) Source M42 — MARKS 7

Explain how a one-tape TM with multiple tracks and state storage can simulate a multitape TM.

2012–2013 • Q3(a) Source M17 — MARKS 8

Prove that every multitape TM has an equivalent single-tape TM.

SOLUTION

SCOPE *2019–2020 Q10(b) and 2012–2013 Q3(a) ask only for the simulation (from "Simulation by one tape" on); the definition of a multitape TM is the extra part of 2023–2024 Q10.*

Multitape TM. A TM with a finite control and k tapes, each with its own independently moving head. Initially the input is on tape 1 with head 1 on its first symbol, all other cells of all tapes hold the blank B, and the control is in the start state. One move depends on the state and the k scanned symbols: $\delta : Q \times \Gamma^k \to Q \times \Gamma^k \times \{L, R, S\}^k$ — it changes state, writes a symbol on every tape and moves each head left, right or not at all (S). Like the one-tape TM it accepts by entering an accepting state.

Simulation by one tape with $2k$ tracks (slides). Let M have k tapes. The single tape of N is divided into $2k$ tracks, so each cell holds a $2k$-tuple (a symbol of the larger alphabet $(\Gamma \times \{B, *\})^k$). For each tape i of M: one track holds tape i's contents, the other track holds a single marker $*$ under the position of head i.

head 1			*			
tape 1	a	b	b	a	B	B
head 2	*					
tape 2	1	0	1	B	B	B

One move of M is simulated by N as follows.

(i) *Gather.* N sweeps from the leftmost head marker to the rightmost. Each time it passes a head marker $*$ of tape i it records the symbol on the contents track of tape i in its finite control ("storage in the state"). It also keeps in its state the number of markers already visited, so it knows when all k have been seen and does not get lost. The state of N thus contains M's state plus the k scanned symbols — finitely many possibilities.

(ii) *Decide.* N now knows the whole argument of δ_M, hence the move: new state, k symbols to write, k directions.

(iii) *Update.* N revisits each head marker, overwrites the tape symbol on the contents track and moves the marker one cell left or right (or leaves it), then records M's new state in its own state.

(iv) The accepting states of N are those that record an accepting state of M: N accepts exactly when M does (and when M has no move, neither has N).

Correctness and cost. The tracks always mirror M's tapes and head positions, so $L(N) = L(M)$. After n moves of M the head markers are at most $2n$ cells apart, so one simulated move costs at most $4n + 2k = O(n)$ moves of N, and n moves cost $O(n^2)$: a single-tape TM is only polynomially slower. Hence every multitape TM has an equivalent single-tape TM, and multitape TMs accept exactly the recursively enumerable languages. ■

25. 2019-20 · Q10(b) · 7 marks · ★★★ High · Theory · L16

Same question — solution already given under #24.

26. 2022-23 · Q11 · 15 marks · ★★★ High · Theory · L18

REPEATED ×4 Same question in 4 papers — stacked below, solved once.

2022–2023 • Q11(i–ii) Source M52 — MARKS (i) 8; (ii) 7

Justify true or false: (i) a deterministic TM can simulate n moves of a conventional computer in polynomial $P(n)$ moves; (ii) a conventional computer can simulate n moves of a nondeterministic TM in polynomial $P(n)$ moves.

2021–2022 • Q10(i–ii) Source M48 — MARKS (i) 7; (ii) 8

Justify true or false: (i) a one-tape TM with multiple tracks can simulate a multitape TM; (ii) a conventional computer can simulate n moves of a nondeterministic TM in polynomially many $P(n)$ moves.

January 2020 Term • Q10(i–ii) Source M38 — MARKS (i) 7; (ii) 8

Explain true or false: (i) a one-tape TM with multiple tracks and state storage can simulate a multitape TM; (ii) a deterministic TM can simulate n steps of a conventional computer in $P(n)$ steps for a polynomial P.

2017–2018 • Q10(i–iii) Source M34–M35 — MARKS 15 (shared)

Explain whether each is true or false: (i) A one-tape TM with multiple tracks can simulate a multitape TM. (ii) A deterministic TM can simulate n moves of a nondeterministic TM in $P(n)$ moves for a polynomial P. (iii) A deterministic TM can simulate n moves of a conventional computer in $P(n)$ moves for a polynomial P.

SOLUTION

SCOPE *2017–2018 asks all three statements A, B, C below; each of the other papers asks two of them (see the table at the end). "Conventional computer simulates an NTM" and "DTM simulates an NTM" are answered together in C, since a DTM and a computer simulate each other with polynomial overhead.*

A. "A one-tape TM with multiple tracks (and state storage) can simulate a multitape TM." — TRUE

Use $2k$ tracks: for each tape one track for contents and one for the head marker. Each move of the k-tape machine is simulated by a sweep that collects the k scanned symbols into the finite control and a sweep that writes and moves the markers (full construction in the previous solution). n moves cost $O(n^2)$ moves.

B. "A DTM can simulate n moves of a conventional computer in $P(n)$ moves." — TRUE

(Slides / Hopcroft–Motwani–Ullman §8.6.) Assume (i) each instruction produces a word at most one bit longer than its arguments (multiplication is done by repeated addition and shifting) and (ii) each instruction on words of length k can be done by a multitape TM in $O(k^2)$ steps. A multitape TM keeps: tape 1 = memory as \$ followed by address$*$value# pairs in binary; tape 2 = instruction counter; tape 3 = memory address (or the value found there); further tapes for the computer's input and for scratch work. If c is the longest word or address in the program and d the number of words it occupies, then after n instructions every word and address has at most $c+n$ bits and at most $d+n$ addresses are in use, so tape 1 holds at most $2(c+n+1)(d+n) = O(n^2)$ cells. Each of the fixed number of address look-ups per instruction takes $O(n^2)$ steps, the operation itself takes $O(n^2)$ by (ii), and shifting tape 1 over when a value changes length takes $O(n^2)$. So one instruction costs $O(n^2)$ and n instructions cost $O(n^3)$ steps of the multitape TM; converting it to one tape squares the time: $O(n^6)$ — still polynomial. Conversely, a computer can simulate a TM. Hence "polynomial on a TM" = "polynomial on a real computer", which justifies defining P with TMs.

C. "A DTM / conventional computer can simulate n moves of an NTM in $P(n)$ moves." — NOT KNOWN; believed FALSE

The known simulation (slides) keeps a queue of IDs of the NTM and explores them breadth-first. With at most m choices per move, n moves can reach $1+m+m^2+\cdots+m^n \le (n+1)m^n$ IDs, so the simulation takes exponential time. A polynomial simulation would mean every NTM running in polynomial time has a polynomial-time deterministic equivalent, i.e. NP $\subseteq$ P, hence P = NP. Nobody knows such a simulation and it is widely believed not to exist. So the statement cannot be justified as true; as far as is known it is false (it is equivalent to the open problem P = NP).

Paper	item	answer
2022–23	(i) B; (ii) C	True; not known / believed false
2021–22	(i) A; (ii) C	True; not known / believed false
Jan 2020	(i) A; (ii) B	True; True
2017–18	(i) A; (ii) C; (iii) B	True; not known / believed false; True

27. 2021-22 · Q10 · 15 marks · ★★★ High · Theory · L18

Same question — solution already given under #26.

28. 2020 (Jan term) · Q10 · 15 marks · ★★★ High · Theory · L18

Same question — solution already given under #26.

29. 2017-18 · Q10 · 15 marks · ★★ Medium · Theory · L18

Same question — solution already given under #26.

6 Undecidability — Lecture 7 — Decidability and undecidability

Notes: CSE211_Atif_Undecidability_Notes.pdf

30. 2022-23 · Q12 · 15 marks · ★★★ High · Proof · L20

2022–2023 • Q12 Source M52 — MARKS 5 + 10 = 15

What is the diagonalization language? Prove that it is not RE.

2017–2018 • Q11 Source M35 — MARKS 9 + 6 = 15

Define (i) the diagonalization language; (ii) the universal language; (iii) the halting problem. Show that one of them is not RE.

2007–2008 • Q12 Source M05 — MARKS 10

Define the diagonalization language and prove it is not recursively enumerable.

2006–2007 • Q4(a) Source M01 — MARKS 10

Prove that the diagonalization language is not recursively enumerable.

SOLUTION

Setup. Enumerate binary strings $w_1 = \varepsilon, w_2 = 0, w_3 = 1, w_4 = 00, \ldots$ (the i-th string is w_i where $1w_i$ is i in binary). Every TM over $\{0,1\}$ has a binary code; let M_i be the TM whose code is w_i (if w_i is not a valid code, M_i is a TM with no moves, so $L(M_i) = \emptyset$). Thus $M_1, M_2, \ldots$ lists *all* TMs.
Definitions.

- *Diagonalization language* $L_d = \{w_i \mid w_i \notin L(M_i)\}$: the strings not accepted by the TM whose code they are.
- *Universal language* $L_u = \{\langle M, w\rangle \mid M \text{ accepts } w\}$, the binary strings $\langle M\rangle 111 w$ coding a TM M and an input w accepted by M (Sipser's A_{TM}).
- *Halting problem* $H = \{\langle M, w\rangle \mid M \text{ halts on } w\}$.

Theorem. L_d is not RE.
Proof (diagonalization). Picture the infinite table with entry $(i, j) = 1$ iff $w_j \in L(M_i)$. Row i is the characteristic vector of $L(M_i)$.

	w_1	w_2	w_3	w_4	$\cdots$
M_1	0	1	1	0	
M_2	1	1	0	0	
M_3	0	0	1	1	
M_4	0	1	0	1	
$\vdots$					$\ddots$

L_d is obtained by *complementing the diagonal*: $w_i \in L_d$ iff entry $(i, i) = 0$. Suppose L_d were RE; then $L_d = L(M_k)$ for some k. Ask whether $w_k \in L_d$:

- If $w_k \in L_d$, then by definition of L_d, $w_k \notin L(M_k) = L_d$. Contradiction.
- If $w_k \notin L_d$, then by definition $w_k \in L(M_k) = L_d$. Contradiction.

Either way we have a contradiction, so no TM accepts L_d: L_d is not RE. (In table terms, L_d differs from every row i at column i.) ■
(L_u and H are RE but not recursive — see the next solution; $\overline{L_u}$ is also not RE.)

31. 2017-18 · Q11 · 15 marks · ★★ Medium · Proof · L20

Same question — solution already given under #30.

32. 2023-24 · Q11 · 15 marks · ★★★ High · Proof · L21

REPEATED ×3 Same question in 3 papers — stacked below, solved once.

2023–2024 • Q11 Source N04 — MARKS 15

Give an example of a language that is not RE. Prove the halting problem is not recursive; you may assume that the universal language is not recursive.

2021–2022 • Q11 Source M48 — MARKS 6 + 9 = 15

Define the diagonalization and universal languages. Show that the halting problem is not recursive.

2019–2020 • Q12 Source M43 — MARKS 4 + 11 = 15

What is the halting problem? Prove it is RE but not recursive.

SOLUTION

Definitions / example. $L_d = \{w_i \mid w_i \notin L(M_i)\}$ (diagonalization language), where w_i is the i-th binary string ($1w_i$ is i in binary) and M_i is the TM whose code is w_i ($L(M_i) = \emptyset$ if w_i is not a valid code), is **not RE** (proof above); so is $\overline{L_u}$. The universal language is $L_u = \{\langle M, w\rangle \mid M \text{ accepts } w\}$. The **halting problem** is $H = \{\langle M, w\rangle \mid M \text{ is a TM that halts (accepts or rejects) on input } w\}$.
H is RE. TM U_H (built like the universal TM U) = "on $\langle M, w\rangle$: simulate M on w; if the simulation halts — whether M accepts or not — accept." U_H accepts exactly the pairs where M halts (and keeps running otherwise), so $L(U_H) = H$.
H is not recursive (reduction from L_u). Suppose a decider D_H for H existed. Build D_u:

> D_u = "On $\langle M, w\rangle$: (1) run D_H on $\langle M, w\rangle$. (2) If it rejects (M loops on w), *reject*. (3) If it accepts, simulate M on w — this is guaranteed to halt — and accept iff M accepted."

D_u always halts and accepts exactly L_u, so L_u would be recursive — contradicting the known (assumed) fact that L_u is not recursive. Hence H is not recursive. ■
Why L_u is not recursive (for completeness). If L_u were recursive, so would be $\overline{L_u}$. A TM that changes its input w to $w111w$ (the code of the pair (M_i, w_i) when $w = w_i$) and then runs the decider for $\overline{L_u}$ accepts w_i iff M_i does not accept w_i, i.e. it accepts L_d — contradicting the previous theorem.

REPEATED ×4 Same question in 4 papers — stacked below, solved once.

33. 2021-22 · Q11 · 15 marks · ★★★ High · Proof · L21

Same question — solution already given under #32.

34. 2020 (Jan term) · Q11 · 15 marks · ★★★ High · Theory · L21

January 2020 Term • Q11(a–b) Source M39 — MARKS (a) 7; (b) 8

(a) Define RE and recursive languages. (b) Give examples, with definitions, of (i) a language that is not RE and (ii) an RE language that is not recursive.

SOLUTION

(a) L is *recursively enumerable* if $L = L(M)$ for some TM M: M accepts every $w \in L$, and on $w \notin L$ it either rejects or runs forever. L is *recursive* if $L = L(M)$ for a TM M that *halts on every input* (accepts members, rejects non-members). Every recursive language is RE; not conversely.
(b)(i) $L_d = \{w_i \mid w_i \notin L(M_i)\}$, where w_i is the i-th binary string and M_i the TM with code w_i. It differs from every $L(M_i)$ on the string w_i, so it is not RE.
(b)(ii) $L_u = \{\langle M, w\rangle \mid M \text{ accepts } w\}$. RE: the universal TM accepts it by simulating M on w. Not recursive: a decider for L_u would give a decider for $\overline{L_u}$ and hence a recogniser for L_d, impossible.

35. 2019-20 · Q11 · 15 marks · ★★★ High · Theory · L21

2019–2020 • Q11(a–e) Source M42 — MARKS 15 (shared)

Give one example in each class: (a) regular; (b) context-free but not regular; (c) recursive but not context-free; (d) RE but not recursive; (e) not RE.

SOLUTION

(a) $\{w \in \{0,1\}^* \mid w \text{ ends in } 01\}$, or 0^*1^*.

(b) $\{0^n1^n \mid n \geq 0\}$ (CFG $S \to 0S1 \mid \varepsilon$; nonregular by pumping).

(c) $\{a^nb^nc^n \mid n \geq 0\}$ (decided by a TM; not context-free by the CFL pumping lemma).

(d) $L_u = A_{TM} = \{\langle M, w\rangle \mid M \text{ accepts } w\}$, or the halting problem H.

(e) L_d (diagonalization language), or $\overline{L_u}$.

36. 2019-20 · Q12 · 15 marks · ★★★ High · Proof · L21

Same question — solution already given under #32.

7 Complexity — Lectures 8–9 — P, NP, NP-completeness, PSPACE

Notes: CSE211_Atif_Complexity_Notes.pdf

37. 2020 (Jan term) · Q1(a) · 6 marks · ★★★ High · Proof · L23

January 2020 Term • Q1(a) Source M36 — MARKS 6

Given an instance of Dominating Set, describe how a certificate can establish that the problem is in NP.

SOLUTION

Instance $\langle G, k\rangle$: does $G = (V, E)$ have a dominating set $D \subseteq V$ with $|D| \leq k$ (every vertex is in D or adjacent to a vertex of D)?
Certificate: the set D itself (a list of at most $k \leq |V|$ vertices — polynomial size).
Verifier $V(\langle G, k\rangle, D)$: (1) check $D \subseteq V$, no repeats, $|D| \leq k$ — $O(|V|)$; (2) mark every vertex of D and every neighbour of a vertex in D by scanning the edge list — $O(|V| + |E|)$; (3) accept iff all vertices are marked. This runs in polynomial time; a yes-instance has a certificate that makes V accept and a no-instance has none. Hence Dominating Set $\in$ NP (it is in fact NP-complete, by reduction from Vertex Cover).

38. 2023-24 · Q13 · 15 marks · ★★★ High · Theory · L24

2023–2024 • Q13 Source N04 — MARKS 8 + 7 = 15

Explain how Cook's theorem shows every problem in NP reduces to SAT. A nondeterministic TM has $\delta(q, t) = \{(q_1, t_1, R), (q_2, t_2, L)\}$. Construct its corresponding Boolean-formula rule under Cook's construction when the head is on cell i at step j. Assume the head is not on the leftmost or rightmost nonblank cell.

2022–2023 • Q13(i) Source M52 — MARKS 8

Explain how Cook's theorem shows every NP problem reduces to SAT in polynomial time.

2021–2022 • Q12(i) Source M48 — MARKS 8

Explain how Cook's theorem shows that every problem in NP reduces to SAT in polynomial time.

SOLUTION

SCOPE *2022–2023 Q13(i) and 2021–2022 Q12(i) ask only for Part 1.*

Part 1: Cook's theorem and the reduction to SAT

Cook's theorem: SAT is NP-complete. SAT$\in$NP is easy (guess a truth assignment and evaluate the formula in polynomial time). The main part is to show that *every* $L \in$ NP reduces to SAT in polynomial time.
Let $L = L(M)$ for a single-tape NTM $M = (Q, \Sigma, \Gamma, \delta, q_0, B, F)$ that makes at most $p(n)$ moves along any branch on an input of length n. If M accepts w ($|w| = n$), there is a sequence $\alpha_0 \vdash \alpha_1 \vdash \cdots \vdash \alpha_k$, $k \leq p(n)$, where α_0 is the initial ID and α_k has an accepting state; in $p(n)$ moves only cells $0, \ldots, p(n)$ can be visited. We map (M, w) to a Boolean formula $F(M, w)$ that is satisfiable iff some computation branch accepts: a truth assignment describes one branch.
Variables ($0 \leq i, j \leq p(n)$, $q \in Q$, $t \in \Gamma$):

- $s(q, j)$: M is in state q during step j;
- l_{ij}: the tape head is on cell i during step j;
- $c(i, j, t)$: cell i contains symbol t during step j;
- $u(i, j)$ ($j < p(n)$): cell i is unchanged from step j to step $j + 1$.

The seven rule families ($\overline{x}$ = NOT x):

Rule 1. Exactly one state per step: $\bigvee_{q\in Q} s(q, j)$, and $s(q, j) \to \bigwedge_{\hat{q}\in Q-\{q\}} \overline{s(\hat{q}, j)}$.

Rule 2. Exactly one head position per step: $\bigvee_{i=0}^{p(n)} l_{ij}$, and $l_{kj} \to \bigwedge_{i\neq k} \overline{l_{ij}}$.

Rule 3. Exactly one symbol per cell: $\bigvee_{t\in\Gamma} c(i, j, t)$, and $c(i, j, t) \to \bigwedge_{\hat{t}\in\Gamma-\{t\}} \overline{c(i, j, \hat{t})}$.

Rule 4. An unchanged cell keeps its symbol: $c(i, j, t) \wedge u(i, j) \to c(i, j + 1, t)$.

Rule 5. The next ID follows by δ: for $\delta(q, t) = \{(q_1, t_1, d_1), \ldots, (q_m, t_m, d_m)\}$,

$$[s(q, j) \wedge l_{ij} \wedge c(i, j, t)] \to \bigwedge_{k\neq i} u(k, j) \wedge \Big[(s(q_1, j{+}1) \wedge l_{\phi(1)(j+1)} \wedge c(i, j{+}1, t_1)) \vee \cdots \vee (s(q_m, j{+}1) \wedge l_{\phi(m)(j+1)} \wedge c(i, j{+}1, t_m))\Big],$$

where $\phi(k) = i - 1$ if $d_k = L$ and $i > 0$; $\phi(k) = i$ if $d_k = L$, $i = 0$ or $d_k = R$, $i = p(n)$; $\phi(k) = i + 1$ if $d_k = R$ and $i < p(n)$.

Rule 6. Initial ID (one formula): $s(q_0,0) \wedge l_{00} \wedge c(0,0,w_1) \wedge \cdots \wedge c(n-1,0,w_n) \wedge c(n,0,B) \wedge \cdots \wedge c(p(n),0,B)$.

Rule 7. Acceptance (one formula): $s(q_a, p(n))$, q_a the accepting state (an accepting branch is taken to stay in q_a once it gets there).

Size. $F(M,w)$ is the AND of all these formulas. Each formula is written directly from its parameters, so $F(M,w)$ is computable from M and w. The largest family is Rule 5: one formula for each i,j,t,q, each containing the $O(p(n))$-long conjunction $\bigwedge_{k\neq i} u(k,j)$, in total $O(|Q|\,|\Gamma|\,p(n)^3)$ — a polynomial in n. So the construction takes polynomial time.

Correctness. An accepting branch $\alpha_0 \vdash \cdots \vdash \alpha_{p(n)}$ gives a satisfying assignment (set each variable as the IDs say). Conversely, in a satisfying assignment Rules 1–3 make every step a well-formed ID, Rule 6 makes step 0 the initial ID, Rules 4–5 make each step follow from the previous one by a move of δ, and Rule 7 makes the last state accepting — an accepting branch. Hence $w \in L \iff F(M,w) \in \text{SAT}$: every NP language reduces to SAT in polynomial time, and with SAT$\in$NP, SAT is NP-complete. Nondeterminism appears only as the $\vee$ in Rule 5: the satisfying assignment chooses the branch.

Part 2: the rule for $\delta(q,t) = \{(q_1,t_1,R),(q_2,t_2,L)\}$

Here $d_1 = R$, $d_2 = L$, and the head is not at either end, so $0 < i < p(n)$, $\phi(1) = i+1$ and $\phi(2) = i-1$. Rule 5 becomes

$$\Big[s(q,j) \wedge l_{ij} \wedge c(i,j,t)\Big] \;\to\; \bigwedge_{k\neq i} u(k,j) \wedge \Big[\big(s(q_1,j{+}1) \wedge l_{(i+1)(j+1)} \wedge c(i,j{+}1,t_1)\big) \vee \big(s(q_2,j{+}1) \wedge l_{(i-1)(j+1)} \wedge c(i,j{+}1,t_2)\big)\Big]$$

"If during step j the machine is in state q with the head on cell i holding t, then every other cell is unchanged, and during step $j+1$ either the state is q_1, cell i holds t_1 and the head is on cell $i+1$ (right move), or the state is q_2, cell i holds t_2 and the head is on cell $i-1$ (left move)." Together with Rule 4, $c(k,j,X) \wedge u(k,j) \to c(k,j+1,X)$ for $k \neq i$ and every $X \in \Gamma$, the unchanged cells keep their symbols. The symbol is written on cell i (where the head was), while the head moves to $i \pm 1$. There is one such formula for every i and j, each of size $O(p(n))$, so this part of $F(M,w)$ has size $O(p(n)^3)$.

39. 2022-23 · Q13(i) · 8 marks · ★★★ High · Theory · L24

Same question — solution already given under #38.

40. 2021-22 · Q12 · 15 marks · ★★★ High · Theory · L24

REPEATED ×2 Same question in 2 papers — stacked below, solved once.

2021–2022 • Q12(ii) Source M48 — MARKS 7

Explain why a polynomial-time algorithm for any NP-complete problem implies P=NP.

January 2020 Term • Q12(a) Source M39 — MARKS 7

State Cook's theorem. Explain why a polynomial-time algorithm for an NP-complete problem implies P=NP.

SOLUTION

SCOPE *2021–2022 Q12(ii) does not ask for the statement of Cook's theorem (first paragraph).*

Cook's theorem: SAT is NP-complete, i.e. SAT$\in$NP and every language in NP reduces to SAT in polynomial time (construction: 2023–2024 Q13 above).
Why a polynomial algorithm for an NP-complete problem gives P $=$ NP**.** L is NP-complete if (1) $L \in$ NP and (2) every $A \in$ NP has a polynomial-time reduction to L. Suppose L is NP-complete and has a deterministic decider running in time $q(m)$. Take any $A \in$ NP, with reduction f computable in time $p(n)$ such that $w \in A \iff f(w) \in L$; note $|f(w)| \leq p(n)$ (a TM writes at most one symbol per move). To decide A: compute $f(w)$, then run L's decider on it — total time $p(n) + q(p(n))$, a polynomial. So $A \in$ P, i.e. NP $\subseteq$ P; with P $\subseteq$ NP (every DTM is an NTM) we get P $=$ NP. ■

41. 2020 (Jan term) · Q12(a) · 7 marks · ★★★ High · Theory · L24

Same question — solution already given under #40.

42. 2017-18 · Q1(a) · 6 marks · ★ Low · Theory · L24

2017–2018 • Q1(a) Source M32 — MARKS 6

Define Boolean Satisfiability with examples.

SOLUTION

A Boolean formula is built from variables, $\wedge$, $\vee$, $\neg$ and parentheses. It is *satisfiable* if some assignment of TRUE/FALSE to its variables makes it true. SAT $= \{\langle\phi\rangle \mid \phi \text{ is satisfiable}\}$; CNF-SAT and 3-SAT restrict ϕ to a conjunction of clauses (with exactly 3 literals per clause for 3-SAT). Examples: $\phi_1 = (x \vee \neg y) \wedge (\neg x \vee y)$ is satisfiable ($x = y = 1$). $\phi_2 = x \wedge \neg x$ is unsatisfiable. $\phi_3 = (x_1 \vee \neg x_2 \vee \neg x_3) \wedge (x_1 \vee x_3 \vee x_4)$ is satisfiable ($x_1 = 1$). SAT is in NP (certificate = assignment) and NP-complete (Cook).

43. 2017-18 · Q1(b) · 20 marks · ★★ Medium · Proof · L25

2017–2018 • Q1(b) Source M32 — MARKS 20

With the knowledge that the 3-SAT problem is NP, prove that graph 3-colorability is NP-complete.

SOLUTION

NOTE The source says "3-SAT is NP"; for the proof we need that 3-SAT is NP-*complete* (Cook–Levin + reduction), which we use.

3-COL $\in$ **NP.** Certificate: a colour for each vertex. Verify in $O(|E|)$ that adjacent vertices differ.
3-SAT $\leq_p$ **3-COL.** Given ϕ with variables $x_1..x_n$ and clauses $C_1..C_m$, build G:

- **(i) Palette:** a triangle on nodes T, F, B (true, false, base). They get three different colours; call the colours by these names.
- **(ii) Variable gadget:** for each x_i, nodes x_i and $\neg x_i$, joined to each other and both joined to B. So one of them is coloured T and the other F — a truth assignment.
- **(iii) Clause gadget** for $C = (a \vee b \vee c)$: an *OR-gadget* OR(u, v) consists of new nodes p, q, o with edges $u-p$, $v-q$, $p-q$, $p-o$, $q-o$. Property: if u, v are both F, then $p, q \in \{T, B\}$ are different, so o is forced to F; if u or v is T, o can be coloured T. Chain $o_1 =$ OR(a, b), $o_2 =$ OR(o_1, c) and join o_2 to both F and B, forcing $o_2 = T$.

G has $3 + 2n + 6m$ nodes and $3 + 3n + 12m$ edges: polynomial.
Correctness. ($\Rightarrow$) If ϕ is satisfied, colour literals by their truth values; in each clause some literal is T, so by the OR property both gadgets can output T, and $o_2 = T$ is consistent with its edges to F, B. ($\Leftarrow$) In any 3-colouring, literal nodes are T/F consistently (variable gadget). If some clause had all three literals F, then $o_1 = F$ and then $o_2 = F$ — but o_2 is adjacent to F. So every clause has a true literal: ϕ is satisfiable.
Since 3-SAT is NP-complete and reduces to 3-COL in polynomial time, and 3-COL $\in$ NP, **3-COL is NP-complete**. ■

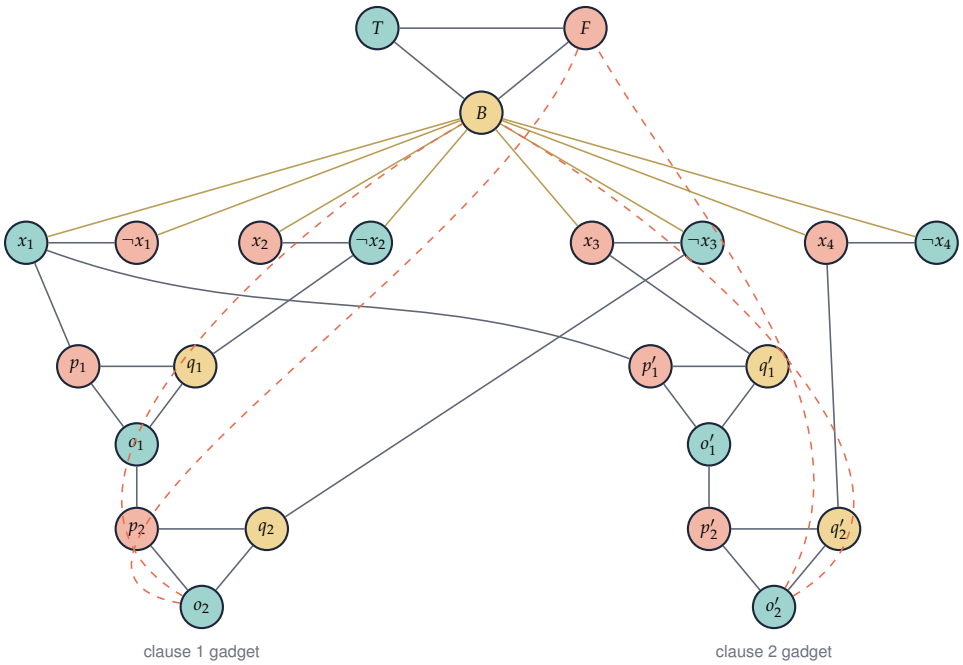
44. 2017-18 · Q1(c) · 9 marks · ★★ Medium · Design · L25

2017–2018 • Q1(c) Source M32 — MARKS 9

Using the graph framework in 1(b), show the final 3-color graph for the 3-SAT formula $(x_1 \vee \neg x_2 \vee \neg x_3) \wedge (x_1 \vee x_3 \vee x_4)$.

SOLUTION

Satisfying assignment used for the colouring: $x_1 = 1$, $x_2 = x_3 = x_4 = 0$. Colours: **T**, **F**, **B**.



Dashed edges join each clause output o_2 to F and B, forcing it to colour T. Every edge joins two different colours, so the graph is 3-coloured, confirming ϕ is satisfiable. (Clause 1: $o_1 = \mathrm{OR}(x_1, \neg x_2)$, $o_2 = \mathrm{OR}(o_1, \neg x_3)$; clause 2: $o'_1 = \mathrm{OR}(x_1, x_3)$, $o'_2 = \mathrm{OR}(o'_1, x_4)$.)

45. 2017-18 · Q12 · 15 marks · ★★ Medium · Proof · L25

2017–2018 • Q12 Source M35 — MARKS 15

The subgraph-isomorphism problem asks whether graph G_1 contains an exact copy of G_2 as a subgraph: choose a subset of G_1's nodes and a one-to-one correspondence with G_2 so the edges among those nodes form the copy. Prove this problem NP-complete.

SOLUTION

SUBISO $= \{\langle G_1, G_2\rangle \mid G_1$ has a subgraph isomorphic to $G_2\}$.
In NP. Certificate: an injective map $f : V(G_2) \to V(G_1)$. Verify in polynomial time that f is one-to-one and that for every edge $(u, v) \in E(G_2)$, $(f(u), f(v)) \in E(G_1)$.
NP-hard: CLIQUE $\leq_p$ SUBISO. CLIQUE $= \{\langle G, k\rangle \mid G$ has a k-clique$\}$ is NP-complete. Map $\langle G, k\rangle \mapsto \langle G_1, G_2\rangle = \langle G, K_k\rangle$, where K_k is the complete graph on k nodes (built in $O(k^2)$ time; if $k > |V(G)|$ output a fixed no-instance).

- If G has a k-clique S, any bijection $V(K_k) \to S$ maps every edge of K_k to an edge of G: K_k is a subgraph of G.
- If K_k embeds into G via f, the k image nodes are pairwise adjacent in G: a k-clique.

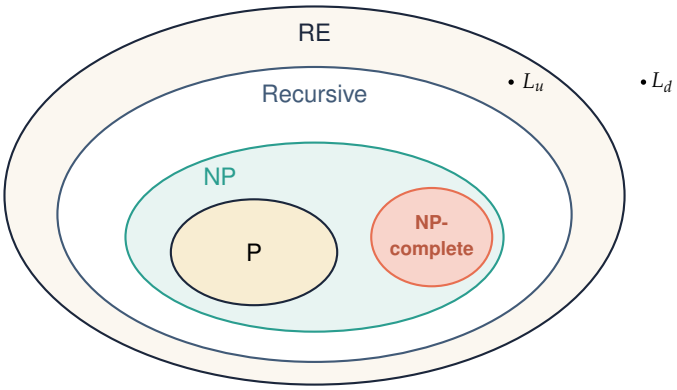
So $\langle G, k\rangle \in$ CLIQUE $\iff \langle G, K_k\rangle \in$ SUBISO. Being in NP and NP-hard, subgraph isomorphism is **NP-complete**. ■ (Under the "induced subgraph" reading the same reduction works, since an induced copy of K_k is also a clique. This is the same idea as the slides' reduction Clique $\leq_p$ Maximum Common Subgraph, where G_2 is a complete graph.)

46. 2023-24 · Q12(i) · 8 marks · ★★★ High · Theory · L26

2023–2024 • Q12(i) Source N04 **MARKS 8**

Draw a Venn diagram of the widely believed relationships among P, NP, NP-complete, recursive, and RE languages.

SOLUTION



- P $\subseteq$ NP: a deterministic TM is a nondeterministic TM that never has a choice. **Believed** P $\neq$ NP (open problem).
- NP-complete problems (e.g. SAT) lie in NP; if P $\neq$ NP none of them is in P (a polynomial algorithm for one of them would give P $=$ NP), so the NP-complete region is drawn inside NP and disjoint from P.
- NP $\subsetneq$ recursive: every branch of a polynomial-time NTM halts within $p(n)$ moves, so trying all branches one by one is an algorithm that always halts. The inclusion is proper: NP is contained in deterministic exponential time, and by the time-hierarchy theorem some decidable languages need more time than that.
- Recursive $\subsetneq$ RE: L_u is RE (universal TM) but not recursive. L_d is not RE, so it lies outside all regions.

47. 2023-24 · Q12(ii) · 7 marks · ★★★ High · Theory · L26

REPEATED ×2 Same question in 2 papers — stacked below, solved once.

2023–2024 • Q12(ii) Source N04 MARKS 7

State the relationship between PSPACE and NPSPACE and briefly justify.

2022–2023 • Q13(ii) Source M52 MARKS 7

State the relationship between PSPACE and NPSPACE, with justification.

SOLUTION

Answer. PSPACE = NPSPACE (Savitch's theorem: $\mathrm{NSPACE}(f(n)) \subseteq \mathrm{SPACE}(f^2(n))$ for $f(n) \geq n$).

PSPACE / NPSPACE are the languages decided by a deterministic / nondeterministic TM that scans at most polynomially many tape cells (for an NTM: on any branch, all branches halting).

PSPACE $\subseteq$ NPSPACE**:** every DTM is an NTM.

NPSPACE $\subseteq$ PSPACE**.** Let NTM N decide L in space $f(n)$.

(i) *Bounded number of moves.* With s states and t tape symbols, N has at most $s\, f(n)\, t^{f(n)}$ IDs using $f(n)$ cells. An accepting computation need never repeat an ID, so if N accepts w it does so within $m = c^{1+f(n)}$ moves ($c = s + t$).

(ii) *Reachability.* $reach(I, J, m)$: "can N go from ID I to ID J in at most m moves?" If $m = 1$, check directly whether $I = J$ or $I \vdash J$. Otherwise try systematically every ID K of size at most $f(n)$ as a midpoint and return true if $reach(I, K, m/2)$ and $reach(K, J, m/2)$ are both true.

(iii) *Space.* The two recursive calls are made one after the other, so only one chain of stack frames $(I_1, J_1, m), (I_2, J_2, m/2), (I_3, J_3, m/4), \ldots$ is on the tape at a time: at most $\log_2 m = \log_2 c^{1+f(n)} = O(f(n))$ frames. A frame holds two IDs ($1 + f(n)$ cells each) and m in binary ($O(f(n))$ cells), i.e. $O(f(n))$ space. Total: $O(f^2(n))$.

(iv) *Deciding w.* With $I_0 = q_0 w$, run $reach(I_0, J, c^{1+f(n)})$ for every accepting ID J using at most $f(n)$ cells (enumerated systematically); accept iff one call returns true.

This deterministic simulation always halts and uses $O(f^2(n))$ space; if f is a polynomial, so is f^2. Hence NPSPACE $\subseteq$ PSPACE, and the two classes are equal. ■

REPEATED ×3 Same question in 3 papers — stacked below, solved once.

48. 2022-23 · Q13(ii) · 7 marks · ★★★ High · Theory · L26

Same question — solution already given under #47.

49. 2021-22 · Q13 · 15 marks · ★★★ High · Theory · L26

2021–2022 • Q13(i–iii) Source M48 MARKS 15 (shared)

Give the widely believed relationships, with brief justifications: (i) P and NP; (ii) P and PSPACE; (iii) PSPACE and NPSPACE.

SOLUTION

(i) P $\subseteq$ NP: a deterministic TM is a nondeterministic TM that never has a choice of moves. **Believed** P $\subsetneq$ NP (open): a polynomial-time NTM can in effect guess one of exponentially many candidate solutions and check it in polynomial time, and no polynomial algorithm is known for any NP-complete problem.

(ii) P $\subseteq$ PSPACE: a TM that makes at most $T(n)$ moves cannot visit more than $1 + T(n)$ cells, so polynomial time implies polynomial space. The same argument gives NP $\subseteq$ NPSPACE, and NPSPACE = PSPACE by (iii); so P $\subseteq$ NP $\subseteq$ PSPACE. **Believed** P $\subsetneq$ PSPACE (open): space can be reused, time cannot.

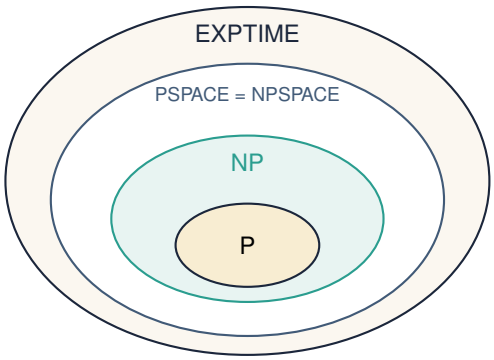
(iii) PSPACE = NPSPACE (**proved**, Savitch's theorem): an $f(n)$-space NTM is simulated deterministically in space $O(f^2(n))$ by the recursive reachability test; see 2023–2024 Q12(ii) above.

50. 2020 (Jan term) · Q12(b) · 8 marks · ★★★ High · Theory · L26

January 2020 Term • Q12(b) Source M39 MARKS 8

Draw a Venn diagram of the widely believed relationships among P, NP, PSPACE, NPSPACE, and EXPTIME, and briefly justify.

SOLUTION



EXPTIME is the class of languages decided by a deterministic TM in time 2^{n^k} for some k.

- P $\subseteq$ NP: a DTM is an NTM. Believed proper.
- NP $\subseteq$ PSPACE: NP $\subseteq$ NPSPACE (in $T(n)$ moves at most $1 + T(n)$ cells are visited) and NPSPACE $=$ PSPACE. Believed proper.
- PSPACE $=$ NPSPACE: proved (Savitch; 2023–2024 Q12(ii) above), so they are one region.
- PSPACE $\subseteq$ EXPTIME: a $p(n)$-space TM with s states and t tape symbols has at most $s\,p(n)\,t^{p(n)}$ IDs, so if it accepts it does so within $c^{1+p(n)}$ moves ($c = s + t$); stopping after that many moves gives an exponential-time decider. Believed proper.

P $\subsetneq$ EXPTIME is *proved* (time hierarchy theorem), so at least one inclusion in P $\subseteq$ NP $\subseteq$ PSPACE $\subseteq$ EXPTIME is strict, but which one is not known.

51. 2019-20 · Q13 · 15 marks · ★★★ High · Theory · L26

2019–2020 • Q13(a–c) Source M43 MARKS 15 (shared)

Explain true or false: (a) polynomial-time SAT implies P=NP; (b) languages decided by polynomial-time nondeterministic TMs are a superset of those decided by polynomial-space deterministic TMs; (c) polynomial-space deterministic and nondeterministic TMs decide the same class of languages.

SOLUTION

(a) True. SAT is NP-complete (Cook's theorem), and a polynomial-time algorithm for an NP-complete problem gives P $=$ NP (2021–2022 Q12(ii) above).

(b) False as stated (not known, and believed false). The statement says NP $\supseteq$ PSPACE. The known relation is the reverse: NP $\subseteq$ NPSPACE (polynomially many moves visit polynomially many cells) and NPSPACE $=$ PSPACE, so NP $\subseteq$ PSPACE. NP $\supseteq$ PSPACE would mean NP $=$ PSPACE, which is believed false.

(c) True: PSPACE $=$ NPSPACE by Savitch's theorem (2023–2024 Q12(ii) above).

52. 2017-18 · Q13(a) · 6 marks · ★★ Medium · Theory · L26

2017–2018 • Q13(a–b) Source M35 **MARKS (a) 6; (b) 9**

(a) Define P, NP, PSPACE, and NPSPACE. (b) Give the widely believed relationship (equality or containment) for (i) P and NP; (ii) PSPACE and NPSPACE; (iii) NP and PSPACE. Briefly justify.

SOLUTION

(a)

- P: $L \in$ P if $L = L(M)$ for some deterministic TM M of time complexity $T(n)$ for a polynomial T (on every input of length n, M halts after at most $T(n)$ moves).
- NP: $L \in$ NP if $L = L(M)$ for a nondeterministic TM M and a polynomial $T(n)$ such that on an input of length n, M has no sequence of more than $T(n)$ moves. Equivalently, membership can be *checked* in polynomial time: $w \in L \iff$ some certificate c with $|c| \leq p(|w|)$ makes a polynomial-time verifier $V(w, c)$ accept.
- PSPACE: languages decided by a deterministic TM whose space complexity (the maximum number of tape cells scanned on an input of length n) is a polynomial.
- NPSPACE: languages decided by a nondeterministic TM, all of whose branches halt, scanning at most polynomially many cells on any branch.

(b)

(i) P $\subseteq$ NP, believed proper (2021–2022 Q13(i) above).

(ii) PSPACE $=$ NPSPACE, proved by Savitch's theorem (2023–2024 Q12(ii) above).

(iii) NP $\subseteq$ PSPACE: NP $\subseteq$ NPSPACE, since a TM making $T(n)$ moves visits at most $1 + T(n)$ cells, and NPSPACE $=$ PSPACE. Believed proper.

16

Out of syllabus

NOTE **The questions in this topic use material that is not covered in Hakim sir's or Atif sir's slides (countability arguments, the formal proof of Rice's theorem, cryptography). They are kept for completeness and solved with the standard textbook method.**

53. 2017-18 · Q13(b) · 9 marks · ★★ Medium · Theory · L26

Same question — solution already given under #52.